

## D7 Individual Reflective Report — Dmytro Chuprynyuk

| Q    | Section       | Key evidence                                                                                           | Rubric                   |
|------|---------------|--------------------------------------------------------------------------------------------------------|--------------------------|
| 1a–c | §1 My role    | 820+ commits; CV lead → infra owner; wk18 sim pivot; Schön BGR debug; 48-h handoff signal              | Specialist; Self-mgmt    |
| 2a–c | §2 My team    | Tuckman mapping (Table 2); wk17 FSM conflict; wk20 complaint letter; Lencioni trust diagnostic         | Team; Decision; M16.c    |
| 3a–b | §3 My impact  | Kolb sim-first cycle; R01–12 traceability; 6-method comms evaluation (Table 3); cost-asymmetry finding | Specialist; M16.d; M17.d |
| 4a–b | §4 My support | Edward/Robin/Demetrio/pilot/Sid named; 3 feedback episodes (Table 4); register lesson; jargon removal  | Insight; M17.c           |

### My role

I want to open with the thing I’m still not comfortable admitting. At the wk12 kickoff I was given a narrow role: train a YOLOv8n dummy detector and hand it to the flight team. By wk20 I owned the state machine, the simulator, the geofence, the GPS estimation pipeline, 127 unit tests, and the D6 bibliography. Nobody asked me to take any of that on. **1a.** The explicit allocation was computer vision — model training, `vision.py` as a standalone module, TFLite export for the Pi. The implicit pickup happened one commit at a time: whenever something sat between the team and a working demo and nobody else was reaching for it, I reached. The shift crystallised around wk16, after our second flight day was cancelled and the project’s centre of gravity moved from the hardware bench to my laptop — because that was the only surface where code could still run. I didn’t announce this; it just happened. I initially framed the drift as diligence. I now see it was partly avoidance: building infrastructure alone felt productive during the roughly eight weeks when three cancelled flight days left us with zero real flight data, and it protected me from the harder skill of slowing down so a teammate could catch up. My Ukrainian engineering training says one person delivers. This project said that’s a failure mode dressed as a strength.

**1b.** What I’m proudest of isn’t the retrained model at  $mAP50 = 0.995$ , or the 820-plus commits, or the D6 score climb from 74 to 85.2 across six scoring cycles. It’s the first cancelled flight day (2026-03-11). The weather killed our only window. Instead of going home I calibrated the focal length to 5.46 mm against a tape measure, ran the checkerboard lens calibration, wrote `FIELD_QUICK_REF.md` for no-internet field use, and closed ten commits before dark. That day crystallised my thesis about this project: *the simulation framework we built ourselves is the only reason this project produced any verifiable engineering output at all*. What I’m proud of is refusing to let a missing drone become a missing report. Had I acted on my “build a simulator first” instinct a week earlier, we’d have saved five days of idle waiting — a pattern I now recognise in myself: I trust the escape route too late.

The BGR camera debugging (2026-02-17) is the incident I want to name as reflection-in-action (Schön, 1983). I spent hours on AWB modes, manual gains, ISP tuning, a kernel update — all wrong. Then I stopped guessing and systematically tested all six channel permutations. Root cause: the IMX296 outputs BGR despite labelling itself RGB888. One reading of that night is “good debugging discipline.” Another is that I burned three hours on intuition-driven fixes before switching to systematic diagnosis, and that ratio tells me something about how I respond to frustration — I reach for plausible explanations instead of exhaustive ones, and the switch only happens after frustration has built up enough to override the comfort of guessing.

**1c.** The distinction I will focus on in future work is between *capability* and *enablement*. I leave this project believing engineering leadership is less about being the one who fixes the geofence sign-convention bug at midnight and more about making my fix legible enough that the module owner would’ve found it without me. The evidence: the `passive_watch_2.py` handoff to Robin (commit `a5b1ab7`). I built the `-robin` flag and `cv_mode` polling interface so his state machine could consume my vision output over HTTP without touching my code. Robin then integrated for two days without asking me a single question. That was the first teamwork move I made on this project that scaled without my presence — and it happened in wk20, not wk12. In my June robotics internship, I’ll schedule two hours per week of explicit pair-programming and handoff review. The falsifiable signal: can a teammate complete a handoff task independently within 48 hours? If not, my documentation has failed, not their attention.

## My team

**2a.** At the wk12 kickoff — credit to Robin for insisting we hold this conversation, I would’ve skipped it — we split on module ownership: Robin took the flight loop and state execution, I took perception and simulation, Edward took the Cube and hardware interface, Demetro took FDR communication, and our PM coordinated schedule. We deliberately declined a single technical lead because the brief devolves autonomy-level choices to the team. The structure drifted twice. The first drift was silent: around wk16, as hardware kept slipping, the project’s centre of gravity migrated to my laptop. Nobody declared this. The second was loud: wk17, after Robin and I discovered we’d been building overlapping state machines in parallel, I asked for a renegotiation. We agreed in writing (`DESIGN_DECISIONS.md:107–109`) that his script would own the flight loop and call my `passive_watch` as an HTTP service. I now see unexplicit structure is a luxury that only survives in slack-rich teams.

**Table 2: Tuckman stage mapping** (Graham/Tuckman, 1965; Jensen, 1977)

| Stage                | Our team evidence                                                                                       | What I did                                                                                                                   | What I’d change                                                            |
|----------------------|---------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Forming (wk12–14)    | Polite role division; trivial questions; looking for leadership that nobody claimed                     | Accepted CV-only scope; didn’t push for interface contracts                                                                  | Front-load a 2-h whiteboard session on inputs/outputs at every seam        |
| Storming (wk15–18)   | wk17 FSM conflict; Robin’s pushback; overlapping code; PM friction after 2nd cancellation               | Proposed 11-state FSM; took 24 h to separate “right design” from “right design for this team”; compromised to HTTP interface | Surface conflicts explicitly in wk15, not wait until overlap is discovered |
| Norming (wk19–20)    | <code>DESIGN_DECISIONS.md</code> contract; Robin integrates unaided; in-jokes about “the missing drone” | Wrote handoff docs; built <code>-robin</code> flag; drafted complaint letter collaboratively                                 | Institute weekly 1-line decision log from wk1                              |
| Performing (wk21–22) | Parallel tracks productive; pilot operates GCS unaided; D6 sections flowing                             | Refactored <code>main.py</code> 1411→754; built 146 pytests; ran R01–12 audit                                                | Pair on refactors instead of soloing                                       |
| Transforming (wk23)  | Lessons-learned doc; sense of loss over untested hardware                                               | Wrote <code>LESSONS_LEARNED.md</code> ; captured 8 reusable patterns                                                         | Share the losses openly, not just the lessons                              |

**2b.** The most impactful team-working element, measured by what it unblocked, was a handoff ratio I hadn’t noticed before: the four hours I spent writing `VISION_PIPELINE_FOR_COLLEAGUE.md` and the `-robin` integration flag bought Robin roughly twenty hours of unblocked integration time across wk20. He wired `passive_watch` into his mission script without asking me a single clarifying question for two days. I now understand those docs worked because they traded my solo time for Robin’s onboarding time at roughly 5:1 — better team economics than I’d assumed documentation ever delivered.

The wk17 FSM conflict needs to be on record. I proposed an 11-state FSM with timeout guards and a geofence repulsor. Robin argued a two-out-of-five maintainability floor meant it was over-engineered; Demetro said the extra states wouldn’t survive a hardware-debug session with Edward. My first reaction was defensive. I took 24 hours, reread Robin’s flight script, and the uncomfortable realisation was that half of my insistence was that the FSM was *my* code. I proposed a compromise on the wk17 call: keep the full FSM for autonomous mode in `main.py`, strip it to lightweight HTTP polling for Robin’s script. He accepted. The integration took an afternoon instead of a week. Using Lencioni’s (2002) framework from our Professional Practice sessions: the conflict was productive because we had just enough trust (Level 1) to disagree openly rather than routing it through back-channels — but I notice I was close to the “fear of conflict” edge, because I needed 24 hours to stop conflating design critique with personal critique.

A second conflict: the wk20 complaint letter about hardware failures. Instead of confronting the PM directly — something I historically do badly — I drafted the letter in measured prose, iterated the tone six times, and shared it only after Robin cut three sentences that were “technically accurate but personally sharp.” The PM disagreed with one paragraph; we rewrote it together. The letter went out

with all five names. What I hadn’t noticed before is that I can only handle confrontation when it’s mediated by writing — and this is a pattern across my personal life, not just this project.

**2c.** Three changes, each falsifiable. First: a weekly one-line decision log from wk1, not wk12 — every call that bound more than one person’s work. The wk16 silent drift would’ve been caught by any written record; signal: zero overlapping-work surprises at integration. Second: interface contracts *before* parallel work — a single wk14 session on the `passive_watch` HTTP contract would’ve prevented the entire wk17 conflict. Third: stop treating documentation as a substitute for conversation. The two pieces of writing that actually changed behaviour were the 400-word pivot message and the complaint letter — both short, timely, paired with a live call. The 3000-word colleague handoff doc sat unread until Robin needed it. Signal for next time: any doc over 500 words gets a 15-minute walkthrough inside a week, or I treat it as unsent.

## My impact

**3a.** The honest thesis is the one I wrote into our group complaint letter: *the simulation framework we built ourselves is the only reason this project produced any verifiable engineering output at all*. I want to defend that with mechanism, not assertion, because at Merit-level it reads as self-flattery.

The anchor is a full Kolb (1984) cycle, used deliberately rather than retrospectively. *Concrete experience*: three collapsed flight days — 2026-03-11 weather cancellation, the 2026-03-30 session that produced no airborne minutes, a third that ended with Edward and me debugging a corrupt `calibration_data.npz` while the pilot stood on a wet field. *Reflective observation*: the team’s entire definition of “progress” had silently assumed hardware access we didn’t have, and everyone was idle because their planned work needed a drone in the air. I was blocked too, but I had a laptop. That asymmetry was the data point. *Abstract conceptualisation*: if `vision.py` exposed a single interface (`detect_in_image` returning (`found`, `x`, `y`, `conf`)) and auto-selected between Ultralytics on laptop and TFLite on Pi, then the same state machine could run in both places — anything proven in simulation would, in principle, run on hardware. *Active experimentation*: six weeks of hammering on that premise. `simple_simulator.py` grew to 2508 lines with GPS drift and camera shake. `main.py` was refactored 1411 → 754 lines across six modules with 146 pytest tests. The `tests/flight/0a-4` ladder plus 127 unit tests gave me a way to know, before any field day, which scripts could survive contact with hardware. When we finally ran on the Pi the delta was thin — a bbox-coordinate bug (commit 3d62e6a), a double-scaling fix (7eb7cc8), a deleted calibration file — and the system came up.

I also unblocked Robin by building a dedicated HTTP integration path (`passive_watch_2.py`, commit a5b1ab7) with JSON schema and `-smart-clear` flag, unblocked Demetro by building his two FDR slides and writing 465 words of speaking script across 4–5 iterations, and drove the team’s external communication via the six-iteration complaint letter.

**Table 3: Communication methods evaluated against 48-h independent-action criterion**

|   | Method & audience                         | Hit? | Author hrs | Lesson                                                                      |
|---|-------------------------------------------|------|------------|-----------------------------------------------------------------------------|
| A | Markdown handoff docs (Robin, wk19)       | Yes  | 4 h        | Scales; but sat unread until needed                                         |
| B | In-person pair walkthrough (Edward, wk20) | Yes  | 1 h        | Compresses clarification loop; doesn’t scale                                |
| C | Plain-language bug report (PM, wk21)      | Yes  | 0.5 h      | Jargon removal: “ACK race” → “drone waiting for a message that never comes” |
| D | Minimal button UI (pilot, wk22)           | Yes  | 3 h        | Non-tech audience: 4-button RC thumb-reach; jargon → plain English          |
| E | FDR speaking script (Demetro, wk16)       | Yes  | 5 h        | My register ≠ his register; good feedback means writing in their voice      |
| F | Complaint letter (course organiser, wk20) | Yes  | 6 h        | Non-engineering reader; 6 tone iterations; Robin cut 3 sharp sentences      |

The finding I didn’t expect: communication effectiveness isn’t about clarity — it’s about *cost asymmetry*. I spent twenty hours on the Markdown doc and Robin spent four integrating. Pair walkthroughs inverted that ratio. The same author-hour budget produced dramatically different behaviour change depending on which method the hours went into.

But the sober closing of 3a: the simulation framework couldn’t tell us what it couldn’t tell us. It couldn’t

predict motion blur at 5 m/s with a real IMX296. It couldn't find the descent stall we accepted as a SITL artefact. It couldn't tell us what GPS timing lag would do to a multi-pass lock. Sim-to-real parity is an argument I can only win about things I modelled, and I was disciplined about that only because I'd been forced to be.

**3b.** Against R01–R12, our system hit 12/12 in simulation but 4/12 on real hardware by flight day. Three of the eight gaps were bench-test items, not engineering failures — coordination was the limiting factor, not effort. I mistook technical leadership for team leadership. They're orthogonal: you can build great tools that don't enable anyone. The evidence: the number of times a teammate other than me edited `simple_simulator.py` or `main.py` was zero. Three mechanisms for my internship. First: every new tool must have a named teammate beneficiary documented *before* it ships — if I can't name them, I don't start. Second: no file over 500 lines gets refactored solo; I pair on the review, and the signal is whether the partner can edit independently within 48 hours. Third: two hours per week of explicit teaching, graded not by whether I explained but by whether the teammate completes a task within 48 hours without a second question.

## My support

**4a.** Four people changed how I worked. A generic thank-you list is a Merit-ceiling move, so I'll name the specific mechanism for each.

**Edward** is the reason the Cube ever spoke to the Pi. Around wk15 I'd spent a weekend trying to make `pymavlink` read from `/dev/ttyAMA0` on Python 3.13. Edward pointed out that `pyserial` reads are broken on 3.13 and the workaround was a `mavproxy` UDP bridge. That 40-minute conversation unlocked roughly two weeks of Pi testing. His throwaway remark — “just because the test passes on laptop doesn't mean the hardware does” — reframed how I thought about simulation fidelity for the rest of the project. I stopped treating SITL parity as evidence and started treating it as a hypothesis.

**Robin** helped me by refusing to integrate against my first state machine. His wk17 pushback is the reason the `main.py` refactor happened at all. What I'd framed as a technical disagreement was actually a maintainability one, and I preferred the technical framing because it protected me from noticing I'd built something only I could read. **Demetro** accepted drafts without turning collaboration into a status contest. **Sid** delivered the single DJI flight video with SRT telemetry that anchored FOV calibration and the retrained model at  $mAP_{50} = 0.995$ .

**4b.** Three specific feedback episodes. “I gave constructive feedback when needed” is the single most common Merit-ceiling sentence in UK engineering reflection, so I won't write it — I'll show the evidence instead.

**Table 4: Feedback given, with behaviour change evidence**

| Recipient      | What I said/did                                                                                                                                                            | Their reaction                                                  | Behaviour change observed                                                                                                        |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Demetro (wk16) | Built 2 FDR slides + 465-word speaking scripts; iterated 4–5 times. On 3rd pass he pushed back: wording wasn't how he spoke                                                | Delivered the slides confidently on FDR day, in his own cadence | A week later prepared his own next slides using specific numbers instead of adjectives — a habit I'd modelled in draft 3         |
| Robin (wk19)   | Sent Teams message naming exact variable ( <code>consecutive_lost</code> ) and line in his flight script; suggested time-based timeout with ascend-and-retry fallback      | Fixed it same afternoon                                         | A week later pulled me aside: he'd caught a similar timeout bug in another module because he was “looking for that shape of bug” |
| Pilot (wk22)   | Showed first <code>pi_flight.py</code> dashboard; he said button grid too dense to hit while holding RC transmitter. Proposed 4-button set (Y/N/M/L) mapped to thumb-reach | Read rewritten card aloud in one pass without pause             | During field-day dry runs, triggered abort 3 times without looking at the screen                                                 |

The Demetro episode taught me something I hadn’t expected about my own delivery: my first three drafts were in my register, not his. Good feedback to a peer presenter means writing in *their* voice, not polishing yours. The scarce resource was register, not prose quality. I now see this connects to Lencioni’s Level 4 (avoidance of accountability): I was holding Demetro accountable to *my* standard of delivery rather than helping him find *his* standard — and the difference matters because accountability that isn’t anchored in the recipient’s own goals produces compliance, not development.

The Robin episode carries a harder lesson about me. My instinct to “finish debugging before asking” isn’t professionalism — it’s a cost I force teammates to pay for my comfort with solitude. I should be asking the moment I notice I’ve been stuck for twenty minutes, not the moment frustration has built up enough that the help-request feels justified to me.

The pilot episode is the non-technical audience adaptation I want to put on record. Two of the five M17.c adaptations landed: jargon removal (“geofence repulsor active within 10 m buffer” → “the drone pushes itself away from the red fence if it gets within 10 m”) and visual substitution (replacing a data table with a four-button panel sized for one-handed RC operation). The rewritten interface was read aloud in one pass — the moment my frame shifted from designing for me-as-operator to designing for a pilot who doesn’t have to notice the UI.

The quieter half of my support work was infrastructure for the team’s memory: `brain-dump.md`, `MEMORY.md`, the 500-line blueprint system, and `FIELD_QUICK_REF.md` for no-internet field use.

**Table 5: Self-development actions with outcomes**

| Gap identified                                              | Action taken                                                   | Outcome                                                                              |
|-------------------------------------------------------------|----------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Debugging by intuition, not system (BGR incident, wk14)     | Adopted exhaustive-permutation-first protocol                  | Applied to TFLite bbox bug (wk22): found in 20 min vs hours                          |
| Solo building without handoff docs (wk12–16)                | Wrote colleague pipeline doc + <code>-robin</code> flag (wk19) | Robin integrated 2 days unaided; 5:1 time trade                                      |
| Conflating “right design” with “right for this team” (wk17) | Took 24 h cool-down before responding to pushback              | wk20 complaint letter: Robin cut 3 sharp sentences, I accepted without defensiveness |
| Treating SITL parity as evidence, not hypothesis (wk15)     | Added explicit sim-vs-real disclaimers to all docs             | Caught descent-stall bug as SITL-specific; didn’t chase it on Pi                     |

**What I leave believing.** I came as a Ukrainian-trained engineer whose reflex under failure was “one person delivers.” I leave believing its near-inversion: *the engineer who delivers alone is one recruitment away from being the single point of failure in whatever she builds*. The wk17 FSM conflict and Robin’s wk20 unaided handoff told me, in different registers, that the same act which delivered the report was the act that kept my teammates at the edge of their own project. Falsifiable internship signal: in my first six weeks, does at least one teammate modify a module I own without asking permission? If yes — even once — I’ve started to become an engineer whose work enables other people’s work. That is the only kind of engineering I now think is worth aspiring to, and I haven’t resolved it yet. The tension between building fast and building together is still live in me, and I don’t expect a six-week internship to settle it. But I know what the test looks like now, and that’s the thing this project gave me.